Color Theory & Accessibility

Workshop 1 — Module 4

Prof.Asoc. Endri Raco

2025/26

Data Visualization for Data Scientists
UNYT Tirana

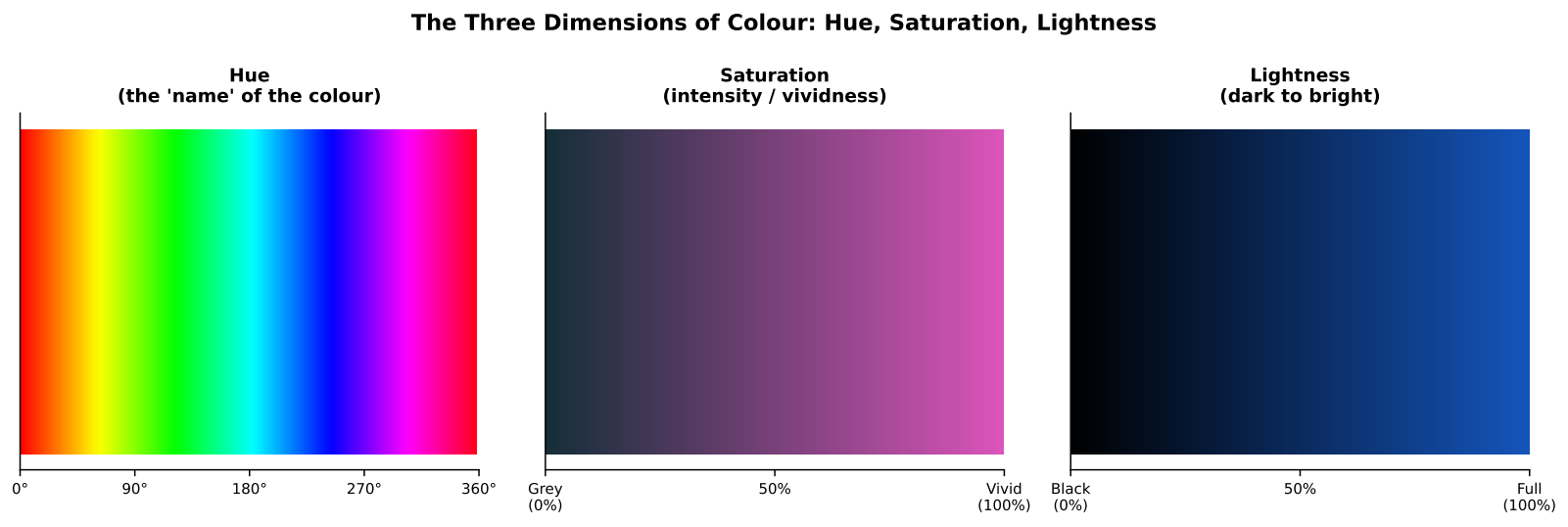
Module Roadmap

1. The three dimensions of colour: hue, saturation, lightness
2. Palette typology: sequential, diverging, qualitative
3. Perceptual uniformity and the viridis family
4. Colour vision deficiency and accessible design
5. WCAG contrast standards and the grey-plus-accent strategy

Learning Outcome
You will be able to select, justify, and implement colour palettes that are perceptually accurate, colourblind-safe, and WCAG-compliant.

The Three Dimensions of Colour

Hue, Saturation, Lightness



- **Hue**: the colour name (red, blue, green) — categorical channel
- **Saturation**: vividness vs. grey — ordinal / emphasis channel
- **Lightness**: dark to bright — sequential / quantitative channel

Mapping Data to Colour Dimensions

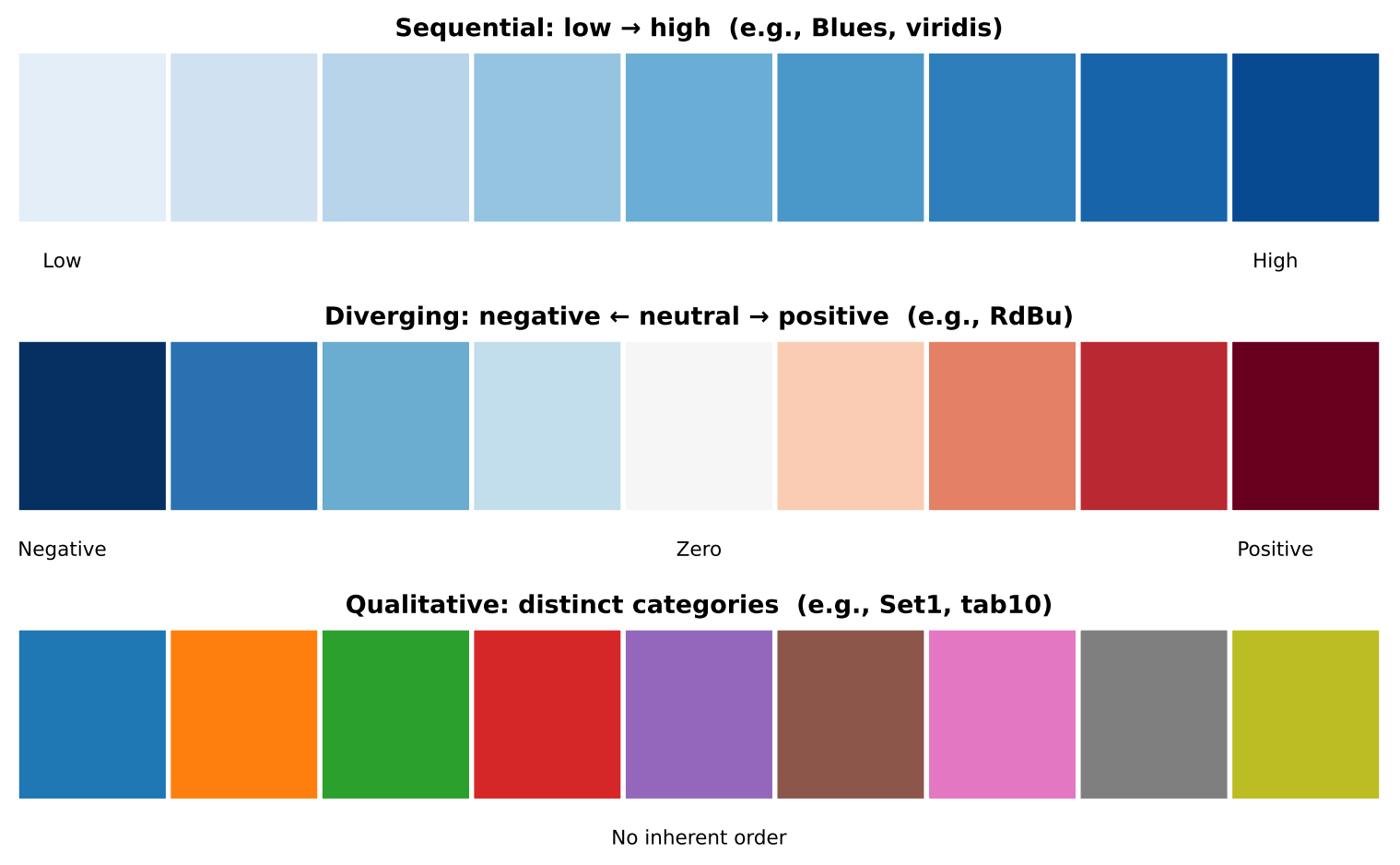
Data Type	Use	Avoid
Nominal (categories)	Hue differences	Lightness gradient
Ordinal (rank)	Lightness / saturation	Hue rainbow
Quantitative (value)	Lightness gradient	Discrete hues
Diverging ($\pm$)	Two-hue + neutral centre	Single hue

Pro-Tip

The single most common colour error is using a **rainbow** (hue variation) for quantitative data. Rainbow palettes create false perceptual boundaries where the data are smooth.

Palette Types

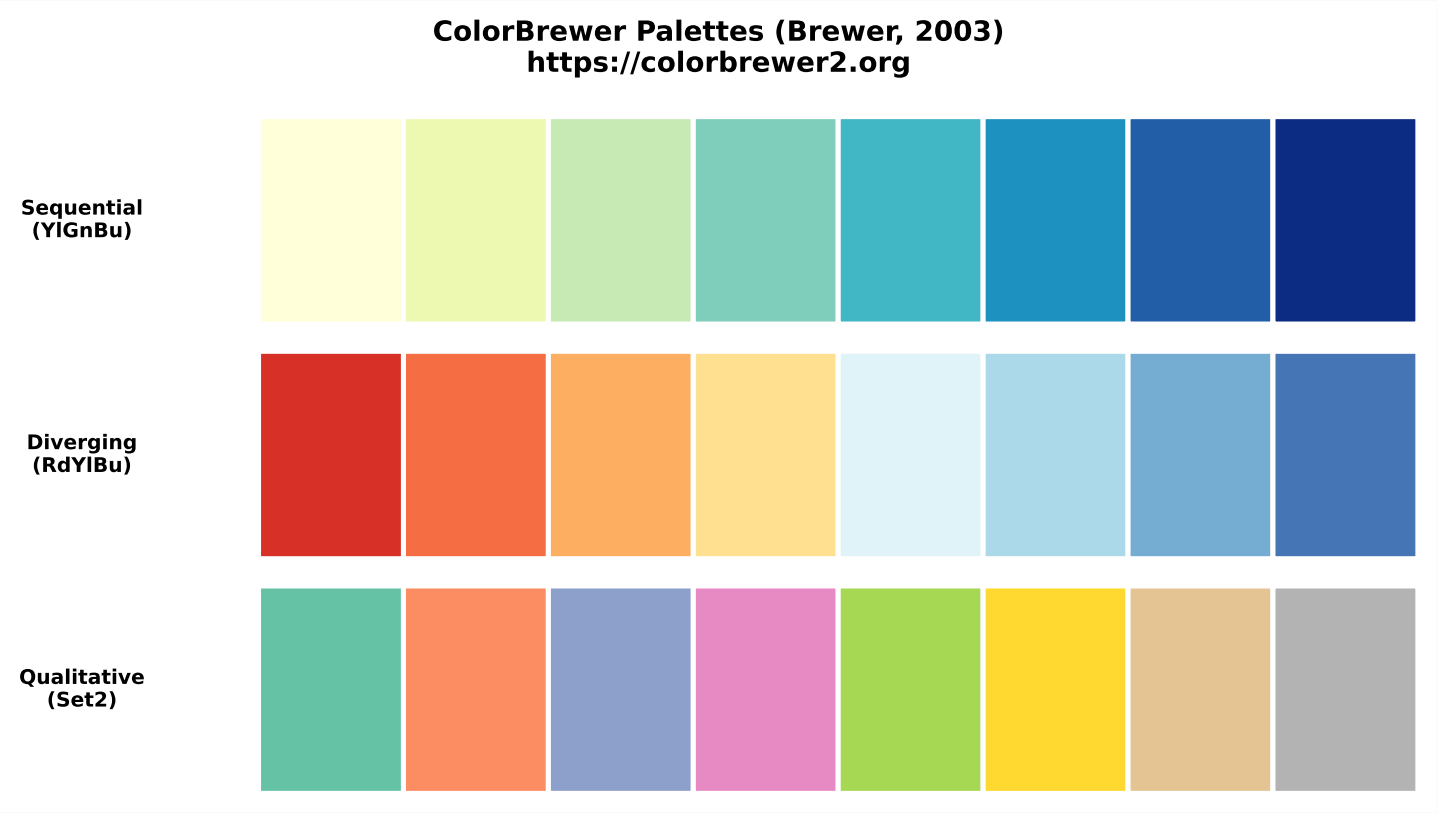
Sequential, Diverging, Qualitative



Data Insight

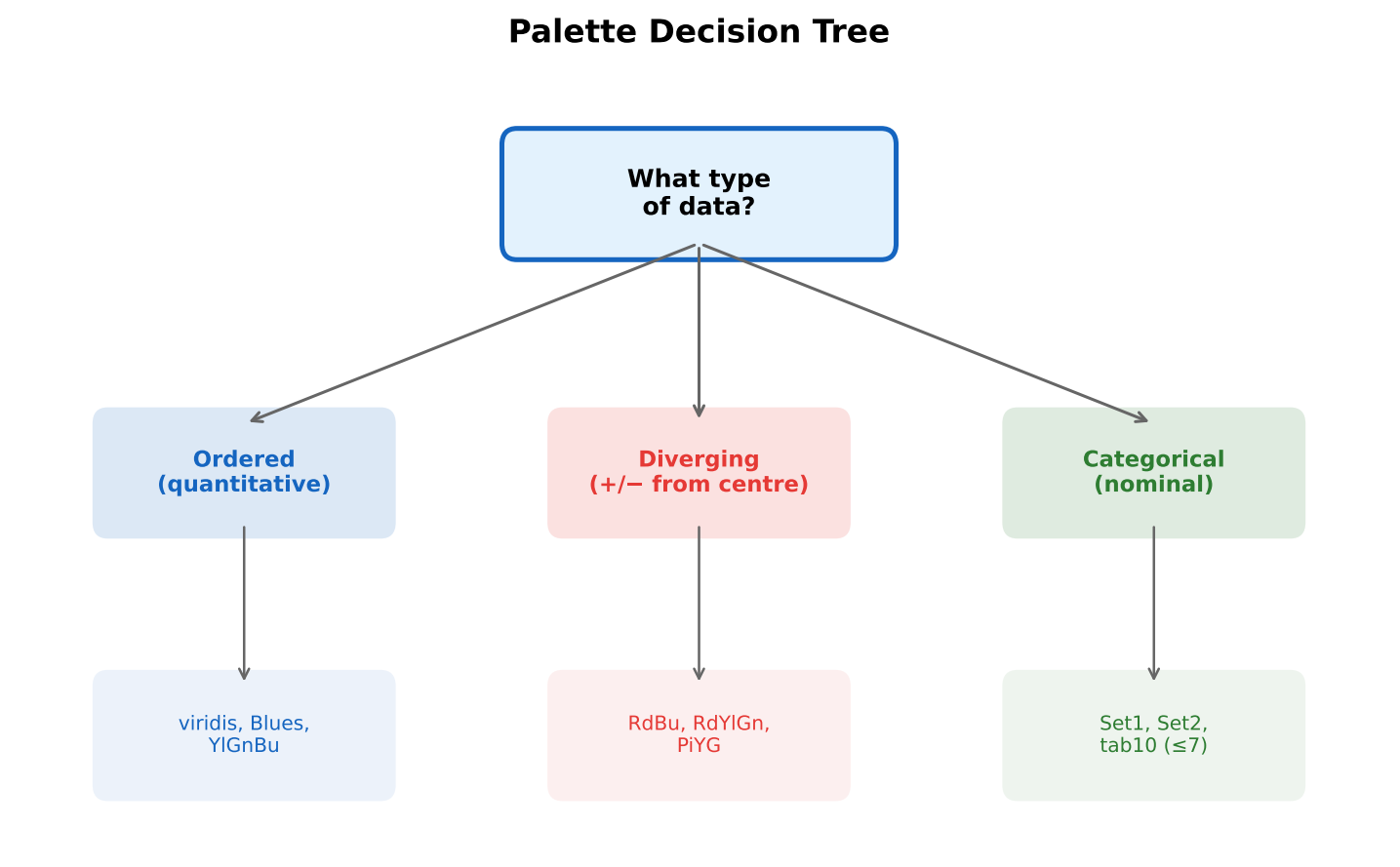
Choosing the wrong palette type is a semantic error: a diverging palette implies a meaningful midpoint; a qualitative palette implies no ordering. Match the palette structure to the data structure.

ColorBrewer: Empirically Tested Palettes



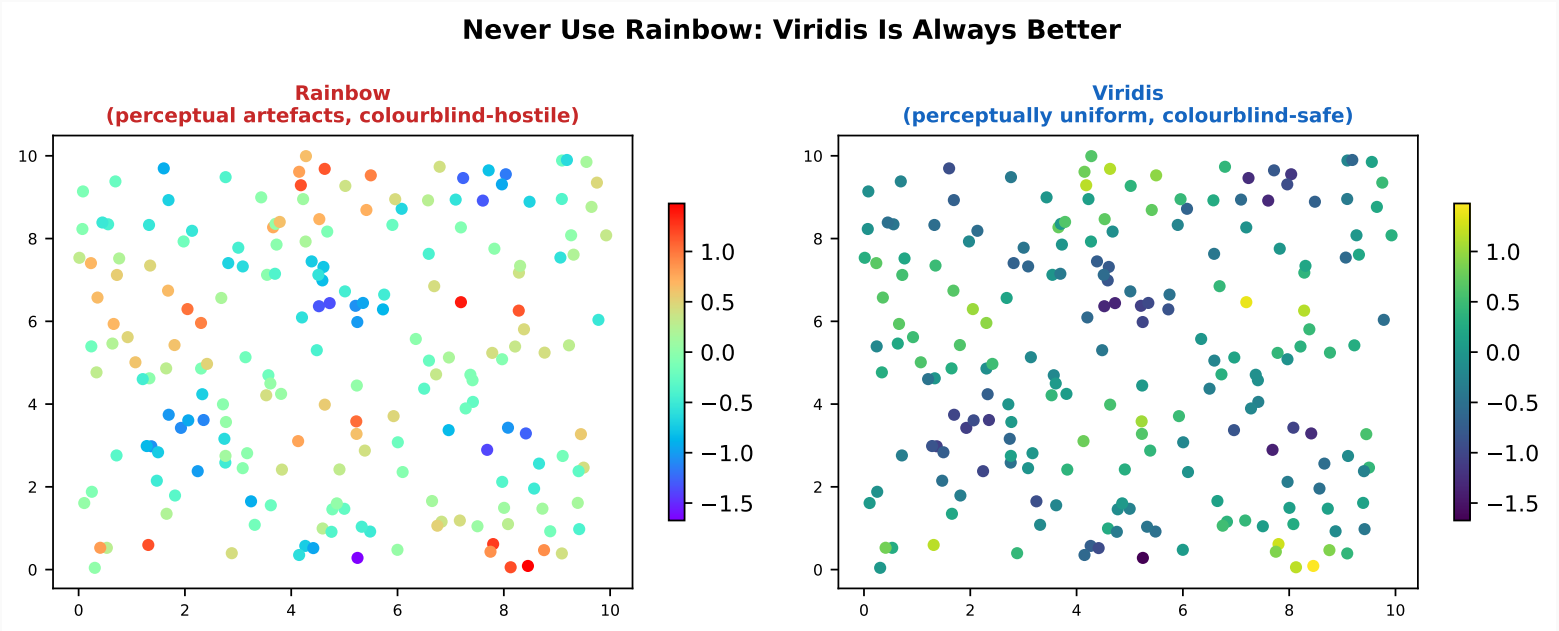
- Brewer (2003) tested palettes for cartographic legibility
- Three families: sequential, diverging, qualitative
- Each family rated for print, photocopy, LCD, and colourblind safety
- Interactive tool: `https://colorbrewer2.org`

The Palette Decision Tree



Perceptual Uniformity

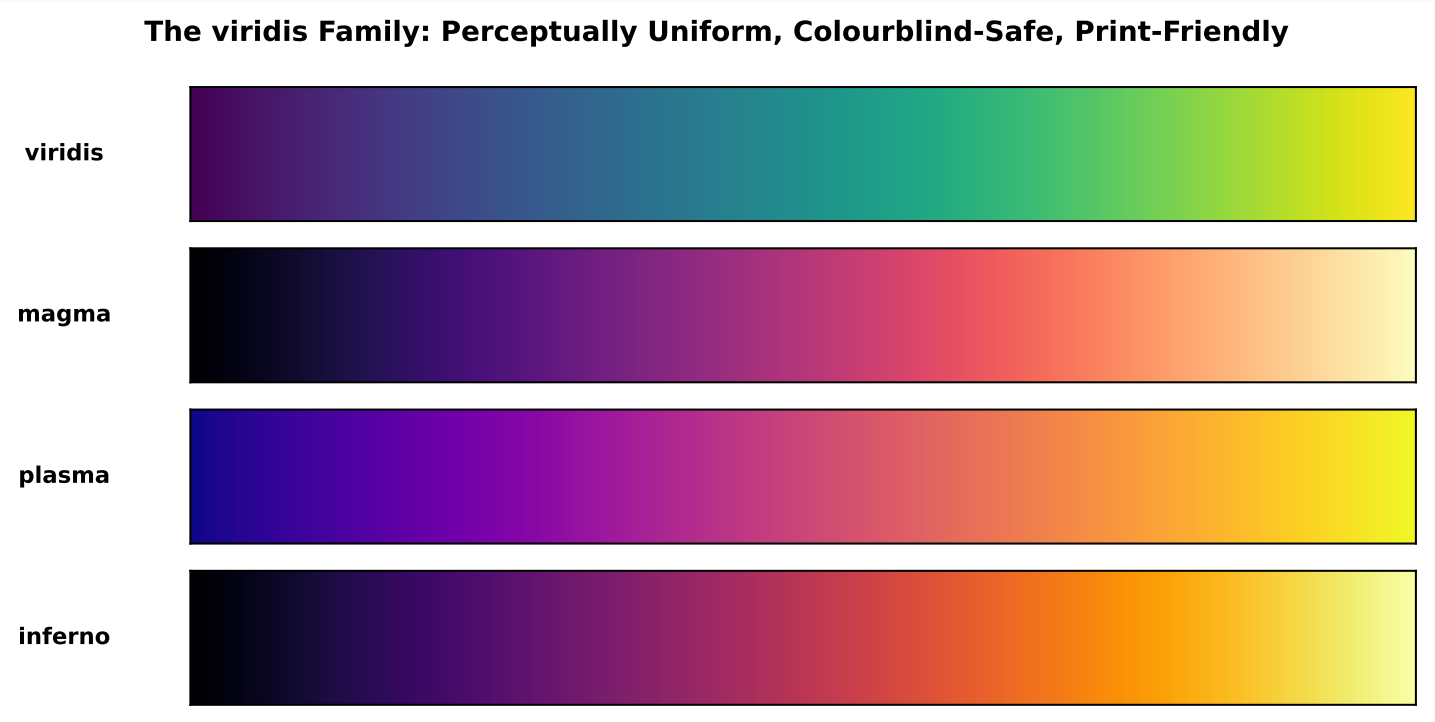
Why Rainbow Fails



Pro-Tip

Rainbow (jet) creates false contour bands at yellow and cyan where the human visual system perceives abrupt lightness jumps. Viridis changes lightness monotonically, so equal data steps produce equal perceptual steps.

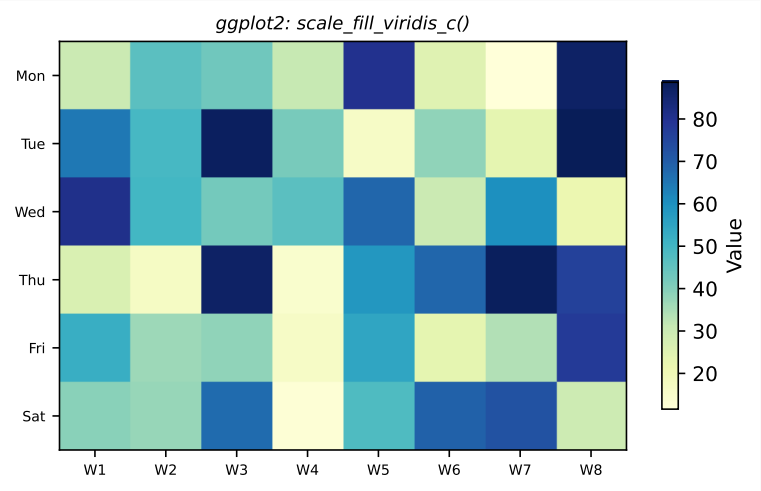
The Viridis Family



- Designed by van der Walt & Smith (2015) for matplotlib
- Perceptually uniform in both colour and greyscale
- Safe for deuteranopia, protanopia, and tritanopia
- Available in R (`viridis`) and Python (`plt.cm.viridis`)

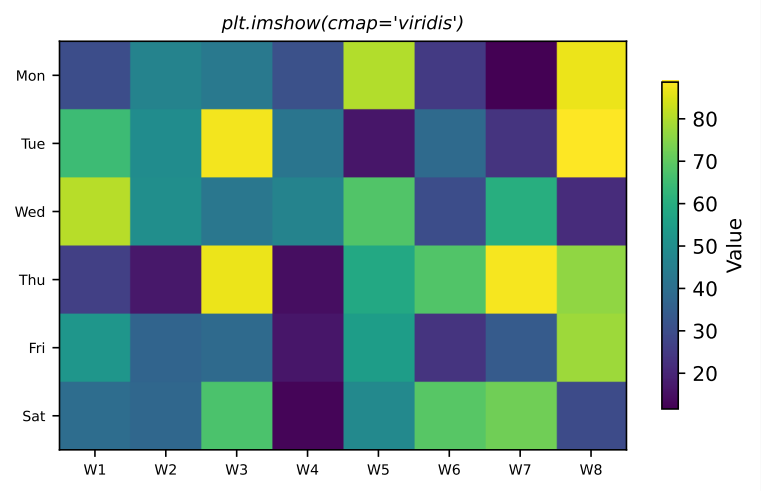
Sequential Palette in R

```r
library(ggplot2)
library(viridis)

# Heatmap with perceptually
# uniform sequential palette
ggplot(df, aes(x = week,
               y = day,
               fill = value)) +
  geom_tile(color = "white",
            linewidth = 0.3) +
  scale_fill_viridis_c(
    option = "D",      # viridis
    direction = 1) +
  theme_minimal(base_size = 9) +
  labs(title = "Weekly Heatmap",
       fill = "Value")

# Alternatives:
# option = "A" (magma)
# option = "B" (inferno)
# option = "C" (plasma)
```



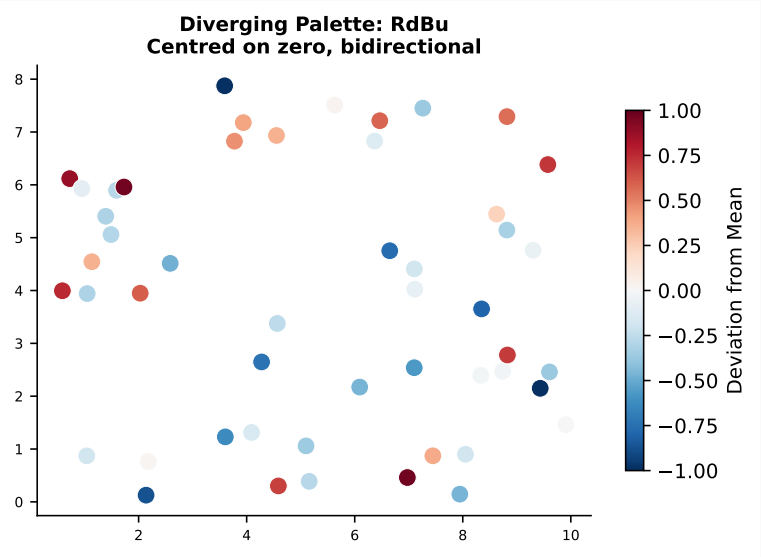
Sequential Palette in Python

```python
import matplotlib.pyplot as plt
import numpy as np

data = np.random.uniform(
    10, 90, (6, 8))

fig, ax = plt.subplots()
im = ax.imshow(
    data,
    cmap="viridis",  # default
    aspect="auto")

plt.colorbar(im, ax=ax,
    shrink=0.8, label="Value")

ax.set_title("Weekly Heatmap")
plt.tight_layout()

# Alternatives:
# cmap="magma"
# cmap="plasma"
# cmap="inferno"
# cmap="cividis" (colourblind)
```



Diverging Palette in R

```r
# Diverging: centred on zero
# Two hues + white midpoint
ggplot(df, aes(x = x, y = y,
               color = deviation))
                    +
  geom_point(size = 2.5) +
  scale_color_gradient2(
    low = "#2166AC",     # blue
    mid = "white",       # neutral
    high = "#B2182B",    # red
    midpoint = 0,
    limits = c(-1, 1)) +
  theme_minimal() +
  labs(color = "Deviation")

# ColorBrewer alternative:
# scale_color_distiller(
#   palette = "RdBu",
#   direction = -1)
```



The midpoint (white) must align with the meaningful centre of the data (usually zero or the mean).

Diverging Palette in Python

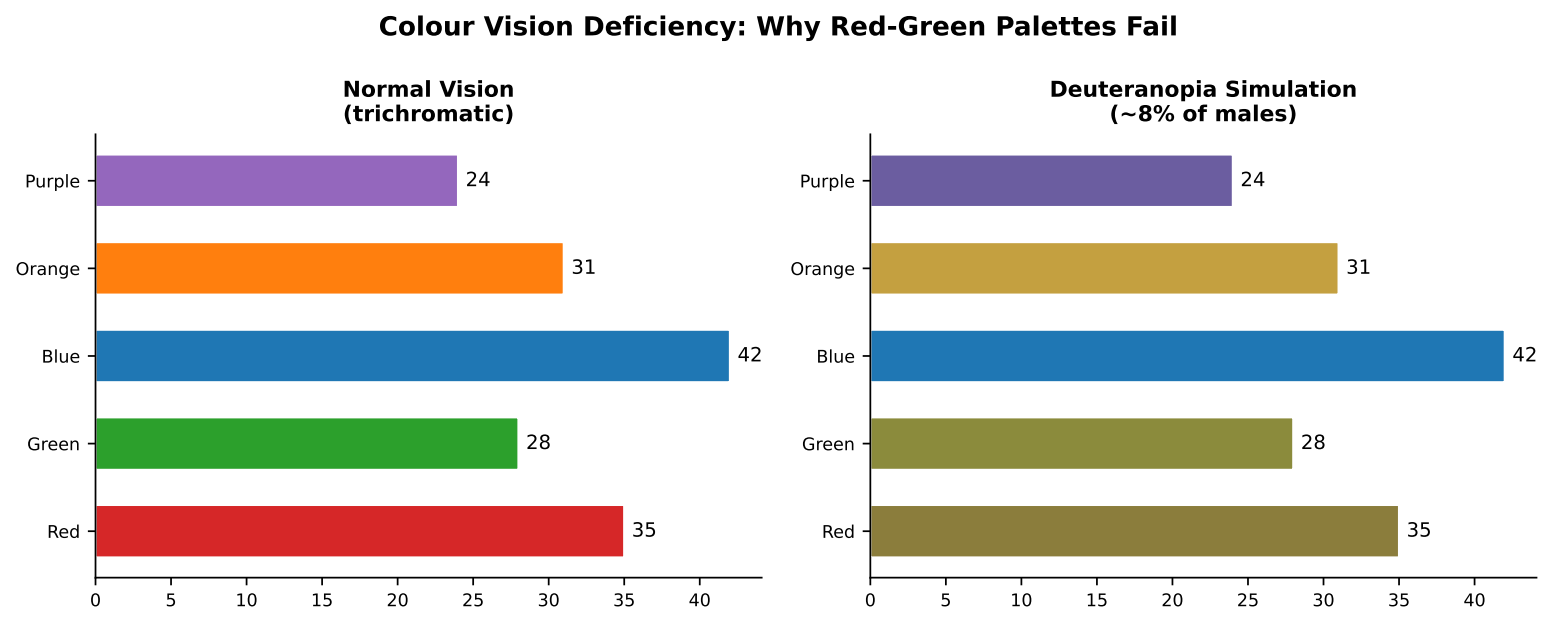
```python
import matplotlib.pyplot as plt

fig, ax = plt.subplots()

sc = ax.scatter(
    x, y, c=deviation,
    cmap="RdBu_r",        # diverging
    vmin=-1, vmax=1,       # centre=0
    s=60,
    edgecolors="white",
    linewidth=0.5)

plt.colorbar(sc, ax=ax,
    label="Deviation from Mean")

# TwoSlopeNorm for off-centre
from matplotlib.colors import (
    TwoSlopeNorm)
norm = TwoSlopeNorm(
    vmin=-2, vcenter=0, vmax=5)
# Pass norm=norm to scatter()
```

Key parameters:

- `vmin`, `vmax`: symmetric around zero
- `TwoSlopeNorm`: for asymmetric ranges (e.g., -2 to $+5$)
- Suffix `_r`: reverses the colourmap direction

Colour Vision Deficiency

8% of Males Have Colour Vision Deficiency



Data Insight

Deuteranopia (red–green deficiency) affects ∼8% of males and ∼0.5% of females. Red and green bars become nearly indistinguishable. Always simulate CVD before publishing.

Accessible Design Strategies

1. Use **viridis** or **cividis** for sequential data — both are colourblind-safe by construction
2. Supplement colour with a **second channel**: shape, pattern, direct label, or line style
3. Simulate CVD using tools:
   - R: `colorspace::deutan()`, `protan()`, `tritan()`
   - Python: `colorspacious` package
   - Web: `https://www.color-blindness.com/coblis/`
4. Limit qualitative palettes to ≤ 7 hues — CVD simulation degrades rapidly beyond this

CVD Simulation in R

```r
library(colorspace)

# Create a plot
p <- ggplot(df, aes(x, y,
    color = category)) +
  geom_point(size = 3) +
  scale_color_manual(
    values = c("A" = "#E53935",
               "B" = "#2E7D32",
               "C" = "#1565C0"))

# Simulate deuteranopia
protan(p)   # protanopia
deutan(p)   # deuteranopia
tritan(p)   # tritanopia

# Better: use safe palettes
p + scale_color_viridis_d()
# or
p + scale_color_brewer(
    palette = "Set2")
```

Workflow:

1. Build chart with default palette
2. Run `deutan(p)` to simulate
3. If categories merge, switch to a colourblind-safe palette
4. Add a redundant channel (shape) as insurance

The `colorspace` package also provides `hcl_palettes()` for systematic palette exploration.

CVD Simulation in Python

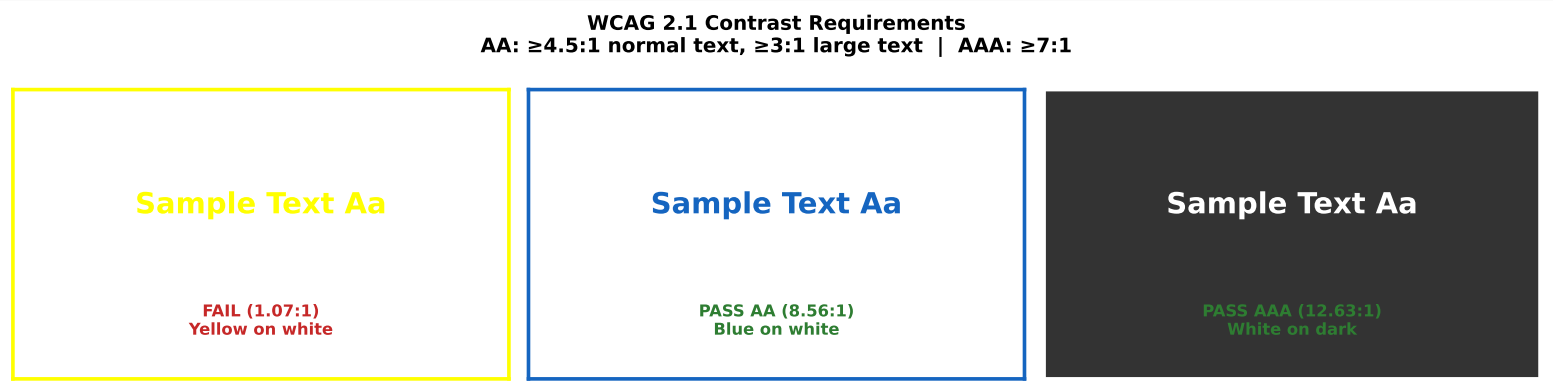
```python
# pip install colorspacious
from colorspacious import
    cspace_convert
import numpy as np

def simulate_cvd(rgb_array,
                 deficiency="
                     deuteranomaly"
                     ,
                 severity=100):
    """Simulate colour vision
    deficiency on an RGB image."""
    # Convert to CVD space
    cvd = cspace_convert(
        rgb_array,
        {"name": "sRGB1",
         "CVD": {
           "type": deficiency,
           "severity": severity}},
        "sRGB1")
    return np.clip(cvd, 0, 1)

# Alternative: use cividis cmap
# plt.scatter(x, y, c=z,
#     cmap="cividis")
# cividis designed for CVD safety
```

Quick alternatives:

- `cmap="cividis"` is optimised for deuteranopia and protanopia
- Seaborn's `palette="colorblind"` provides a 6-colour safe set
- Add markers: `style="category"` in `sns.scatterplot()`

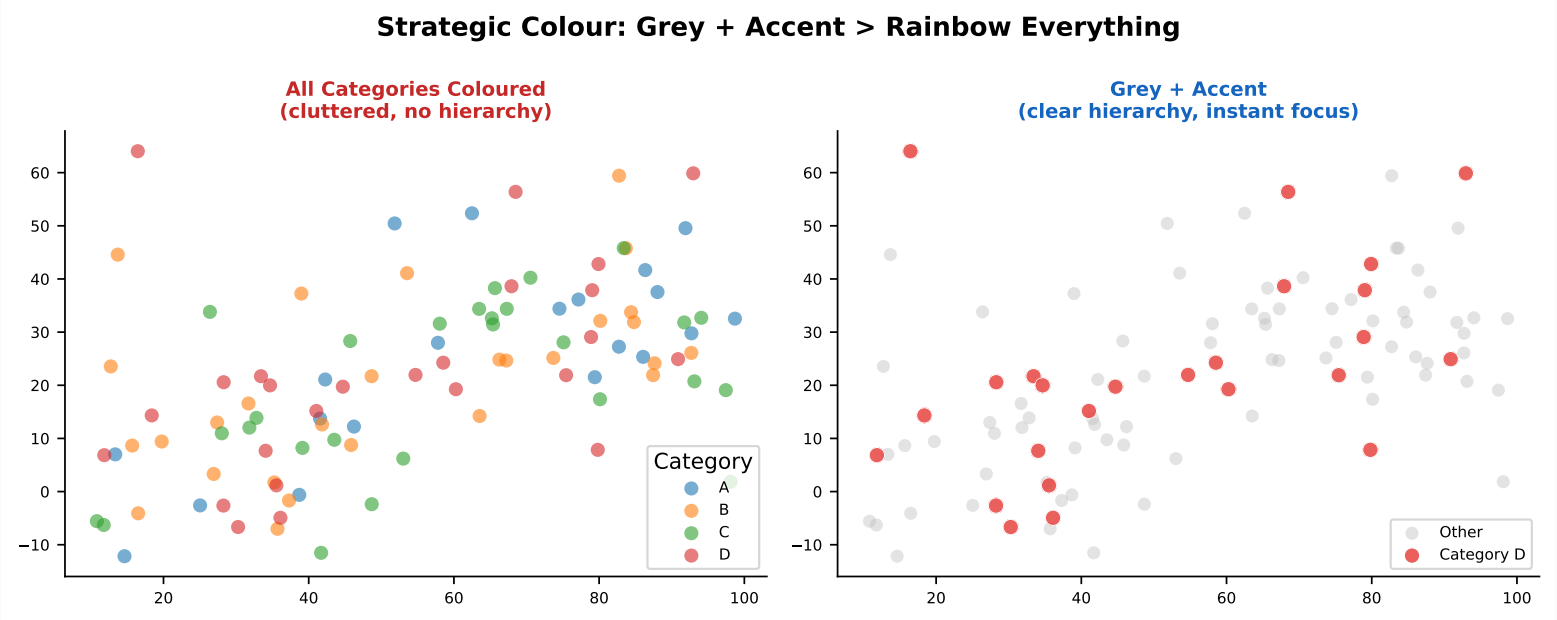
Contrast & the Grey-Accent Strategy

WCAG 2.1 Contrast Requirements



- **AA**: contrast ratio $\geq$ 4.5:1 for normal text, $\geq$ 3:1 for large text (18pt+)
- **AAA**: contrast ratio $\geq$ 7:1
- Check with: `https://webaim.org/resources/contrastchecker/`

Grey + Accent: The Most Powerful Colour Strategy



Data Insight

Colouring everything equally gives everything equal salience — which means nothing stands out. Grey desaturates the background; a single accent hue creates instant pre-attentive pop-out for the focal data.

Grey + Accent in R

```r
library(ggplot2)

# Highlight one category, grey rest
ggplot(df, aes(x = x, y = y,
    color = is_highlight)) +
  geom_point(aes(
    size = ifelse(is_highlight,
                  3, 1.5),
    alpha = ifelse(is_highlight,
                   0.9, 0.3))) +
  scale_color_manual(
    values = c(
      "TRUE" = "#E53935",
      "FALSE" = "#BBBBBB")) +
  scale_size_identity() +
  scale_alpha_identity() +
  theme_minimal() +
  theme(legend.position = "none") +
  labs(title = "Grey + Accent")
```

Three-step recipe:

1. Create a boolean column `is_highlight`
2. Map `TRUE` to accent, `FALSE` to grey
3. Increase size and alpha for the highlighted subset

This leverages both colour and size pre-attentive channels simultaneously.

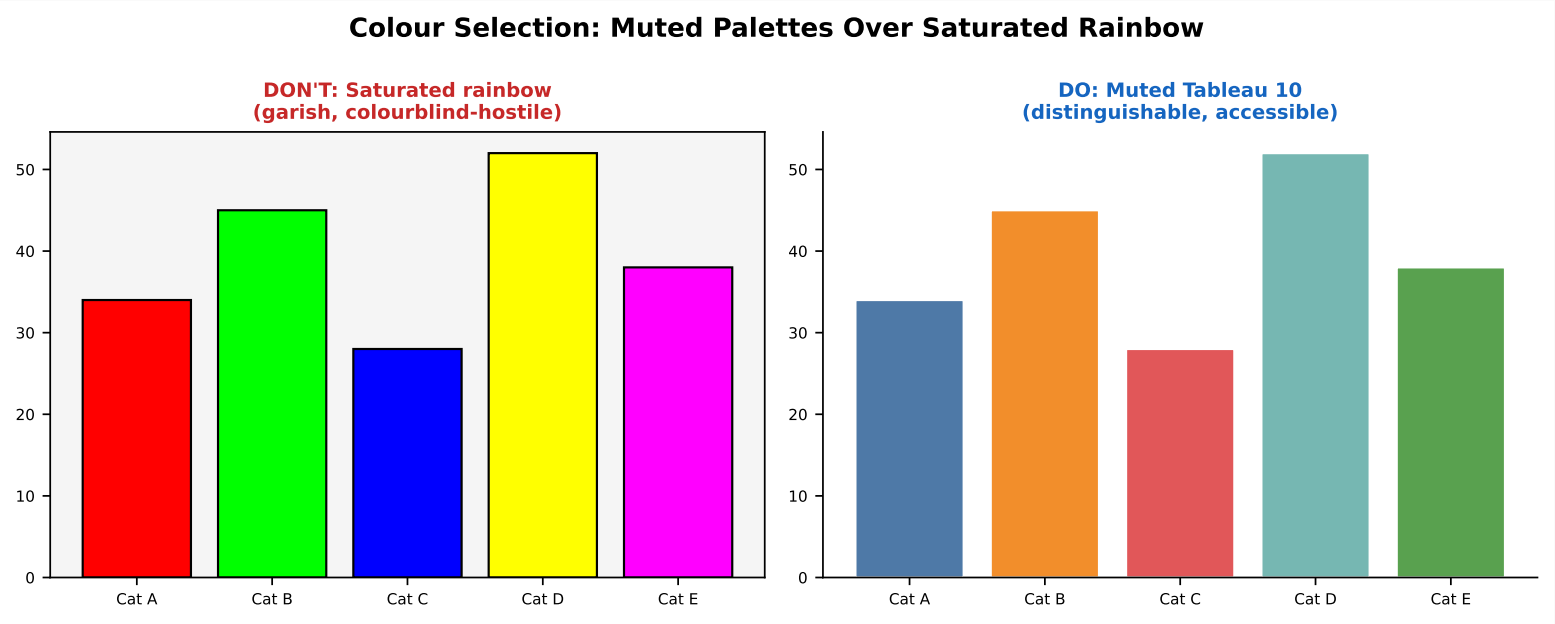
Grey + Accent in Python

```python
import matplotlib.pyplot as plt

fig, ax = plt.subplots()

# Background points (grey)
mask = df["category"] != "target"
ax.scatter(
    df.loc[mask, "x"],
    df.loc[mask, "y"],
    c="#BBBBBB", s=20,
    alpha=0.3, edgecolors="none")

# Highlighted points (accent)
mask_hl = df["category"] == "target
    "
ax.scatter(
    df.loc[mask_hl, "x"],
    df.loc[mask_hl, "y"],
    c="#E53935", s=50,
    alpha=0.9,
    edgecolors="white",
    linewidth=0.5, zorder=5)

ax.set_title("Grey + Accent")
```

Python pattern:

1. Plot background first (grey, small, low alpha)
2. Plot highlight second (accent, large, high alpha, `zorder=5`)
3. No legend needed — the colour difference is self-explanatory

Colour Do's and Don'ts



Synthesis

Key Takeaways

1. **Hue** for categories, **lightness** for quantities, **saturation** for emphasis — never conflate the three.
2. **Sequential** for ordered data, **diverging** for data with a meaningful midpoint, **qualitative** for nominal categories (≤ 7).
3. **Never use rainbow**. Use `viridis`, `magma`, or `cividis` for perceptual uniformity.
4. **Simulate CVD** before publishing (`colorspace` in R, `colorspacious` in Python).
5. **Grey + accent** is the single most effective colour strategy for explanatory visualization.

Essential Readings for This Module

- Brewer, C. A. (2003). "A Transition in Improving Maps: The ColorBrewer Example." *Cartography and Geographic Information Science*, 30(2).
- van der Walt, S. & Smith, N. (2015). "A Better Default Colormap for Matplotlib." SciPy 2015.
- Crameri, F., Shephard, G. E. & Heron, P. J. (2020). "The Misuse of Colour in Science Communication." *Nature Communications*, 11, 5444.
- Wilke, C. O. (2019). *Fundamentals of Data Visualization*, Chapter 4 (Color Scales).

Next Module: Typography, Layout & Composition (W01-M05)